Two-Sample Mendelian Randomization using MR-PRESSO

This notebook demonstrates how to perform a two-sample mendelian randomization using the Mendelian Randomization Pleiotropy RESidual Sum and Outlier (MR-PRESSO) test to evaluate pleiotropy in multi-instrument Mendelian Randomization utilizing genome-wide summary association statistics.

> **Kernel note:** this entire notebook runs in the **R** kernel

Introduction

Mendelian randomization (MR) was developed as a method to help understand the causal effects of modifiable influences (i.e., potential risk factors) on health, social, and economic outcomes, using genetic variants associated with the specific exposures of interest. However, standard MR can fail if invalid instruments introduce horizontal pleiotropy — when a single genetic variant independently influences two or more distinct traits or biological pathways, rather than one trait causing the other. MR-PRESSO detects, corrects for, and tests the significance of the distortion caused by that pleiotropy, extending the standard multi-instrument MR approach for use with summary statistics.

MR-PRESSO has three components:

1. detection of horizontal pleiotropy (MR-PRESSO global test)
2. correction of horizontal pleiotropy via outlier removal (MR-PRESSO outlier test)
3. testing of significant distortion in the causal estimates before and after outlier removal (MR-PRESSO distortion test).

MR-PRESSO software is available as an `R` (version ≥ 3.3.1) package. MR-PRESSO has already been installed in this container for the minimal working example (MWE) below, however local installation instructions can be found on the MR-PRESSO github.

Data

For this exercise, we are performing MR where are exposure is protein expression levels for the protein LRRC37A2 and Sensorineural Hearing Loss (SNHL) is the outcome. Although two-sample Mendelian Randomization is often performed bidirectional. In this example we are only performing two-sample Mendelian randomization in the Forward direction due to time constraint. Where in the backward direction SNHL would be the exposure and protein expression level the outcome.

Protein summary statistics

The summary statistics come from Sun et al. (2023) — the UK Biobank Pharma Proteomics Project — which measured 2,923 plasma proteins (including LRRC37A2) in ~54,000 UK Biobank participants using the Olink Explore 3072 assay, then ran a GWAS on each protein to identify its pQTL (genetic variants associated with that protein's level).

A pQTL is *cis* if it falls within 1Mb of the transcriptional start and stop signs encoding its protein, and *trans* otherwise. This exercise uses only LRRC37A2's genome-wide-significant (p-value ≤ 5×10^{-8}) cis pQTL, since its pQTL heritability is almost entirely cis:

Protein	cis h^2	total h^2	cis share
LRRC37A2	0.658	0.738	~89%

Transcriptional start and stop signs (GRCh38)	Transcriptional start and stop signs (GRCh37)	cis pQTL	total genome-wide significant (p-value ≤ 5×10^{-8}) pQTL
chr17:46,511,508-46,556,910	chr17:43,598,242-45,634,197	6,680	12,367

The data file (`lrrc37a2_5e08_significant_cis_variants.txt`) contains these cis pQTL. While this exercise only uses cis variants, MR-PRESSO can also be run analyzing both cis and trans variants.

MVP sensorineural hearing loss summary statistics

Publicly available summary statistics from a genome-wide association study (GWAS) of sensorineural hearing loss (SNHL) were obtained from the Million Veteran Program (MVP) (Verma et al., 2024) for 401,917 European-American participants (N=191,819 cases, N=210,098 controls) analyzing 19,708,817 variants of which 6,885 variants met genome-wide significance (p-value ≤ 5×10^{-8}) for SNHL.

For this exercise, the file (`MVP_outcome_liftover_lrrc37a2_cis.txt`) contains the MVP sensorineural hearing loss summary statistics restricted to the LRRC37A2's cis window (as defined in the section above) leaving 6,898 variants for analysis of which 2 variants met genomie-wide significance for SNHL.

Genotype data

For **Step 2** (LD clumping) we need a reference panel of individual-level genotypes to measure correlation between candidate SNPs. We use the public 1000 Genomes Project phase 3 EUR panel in a production pipeline you'd instead clump in-sample, against genotypes from the same cohort as the exposure GWAS. Another panel that could be used is from the UK Biobank which is publically available for download here

The panel here (`1000G.EUR.mwe.lrrc37a2.{bed,bim,fam}`) is subset down to just LRRC37A2's cis region (GRCh37 window chr17:43,598,242-45,634,197, via `prepare_ld_panel.sh`), EUR samples (N=489), and biallelic variants (N = 46,126). It also excludes one multi-allelic STR (rs141805917), which the raw panel represented as 3 duplicate `.bim` rows sharing an ID — plink's `--clump` / `--exclude` require unique variant IDs, so this variant is dropped from the reference panel (originals preserved in `./data/original_backup/`):

Candidate instrument pipeline (protein cis pQTL → MR-PRESSO instruments):

Stage	Count
Genome-wide significant pQTL (p-value ≤ 5×10^{-8})	12,367
Cis pQTL (LRRC37A2)	6,680
Overlap with MVP (chr:pos)	5,083
Final instrument set retained after LD clumping (Step 2) and harmonizing with the outcome (Step 3)	115

MR-PRESSO can be run unidirectionally or bidirectionally, depending on whether you want to evaluate pleiotropy in a single assumed pathway (e.g., Exposure → Outcome) or investigate a two-way feedback loop between traits (e.g., Exposure → Outcome & Outcome → Exposure).

An instrumental variable (IV) is a variable that can be used to explore the unconfounded causal effect of an exposure on an outcome. To be valid, an IV must satisfy three core assumptions:

1. **Relevance** — the genetic variant is robustly associated with the exposure of interest.
2. **Independence** (exchangeability) — the variant is not associated with any confounders of the exposure-outcome relationship.
3. **Exclusion restriction** — the variant affects the outcome *only* through the exposure, not through any other pathway.

Horizontal pleiotropy is precisely a violation of assumption 3: a variant affecting the outcome through some pathway other than the exposure being tested.

For this example, we will be running a unidirectional analysis for the purpose of time, in which the Exposure (protein) → Outcome (SNHL). If we were to run a bidirectional MR analysis for this example, the first direction (referred to as *Forward*) would be the same as the unidirectional Exposure (protein) → Outcome (SNHL), and the second direction (referred to as *Reverse*) would test the opposite: Exposure (SNHL) → Outcome (pQTL), comparing the results in both directions.

Direction	Exposure	Instruments	Outcome
Forward (run below)	Protein	Cis pQTL overlapped with MVP, then LD-clumped	SNHL
Reverse (not run here)	SNHL (MVP)	SNHL variants overlapped with protein stats, then LD-clumped	Each protein's own GWAS

Overview

Four steps below:

1. **Overlap** — find genetic instruments for the protein (its significant pQTL) for which the variants are also present in the SNHL GWAS.
2. **LD clumping** — keep only *independent* instruments (MR requires instruments not correlated with each other).
3. **MR-PRESSO** — harmonize exposure/outcome effect estimates onto the same allele, then run the pleiotropy test.
4. **Post-processing** — pick the final causal estimate (outlier-corrected if available, otherwise raw), in the same schema a production pipeline that merges many proteins together.

Requirements

MR-PRESSO is ran as an R package, the necessary dependencies are:

- R with `TwoSampleMR`, `MRPRESSO`, `ieugwasr`, `data.table`, `dplyr`
- A local plink binary and a local 1000 Genomes plink bed/bim/fam reference panel (EUR subset recommended), for Step 2. The panel's `.bim` must use rsIDs.

```
install.packages(c("data.table", "dplyr"))
remotes::install_github("MRCIEU/TwoSampleMR")
remotes::install_github("MRCIEU/ieugwasr")
remotes::install_github("rondolab/MR-PRESSO")
```

Program documentation

- **PLINK**: 1.90 docs
- **R**: r-project.org — from within R, `help()` documents any command
- **MR-PRESSO**: cnsgenomics.com/software/gcta - software installed directly from GitHub and implemented as an R package.

References

- Chang CC, Chow CC, Tellier LCAM, Vattikuti S, Purcell SM, Lee JJ (2015) Second-generation PLINK: rising to the challenge of larger and richer datasets. GigaScience, 4.
- Purcell S, Neale B, Todd-Brown K, Thomas L, Ferreira MAR, Bender D, Maller J, Sklar P, de Bakker PIW, Daly MJ & Sham PC (2007) PLINK: a toolset for whole-genome association and population-based linkage analysis. *American Journal of Human Genetics*, 81:559-575.
- R Core Team (2025), R: A Language and Environment for Statistical Computing, R Foundation for Statistical Computing
- Sun, B.B., Chiou, J., Traylor, M. et al. Plasma proteomic associations with genetics and health in the UK Biobank. Nature 622, 329–338 (2023). https://doi.org/10.1038/s41586-023-06592-6
- Verbanck, M., Chen, CY., Neale, B. et al. Detection of widespread horizontal pleiotropy in causal relationships inferred from Mendelian randomization between complex traits and diseases. Nat Genet 50, 693–698 (2018). https://doi.org/10.1038/s41588-018-0099-7
- Verma A, Huffman JE, Rodriguez A, Conery M, Lui M, et al. Diversity and Scale: Genetic Architecture of 2,068 Traits in the VA Million Veteran Program. Science. 2024 July 19 doi: 10.1126/science.adj1182.

In [1]:
```
# Load the packages
suppressMessages({
  library(data.table)
  library(dplyr)
  library(ieugwasr)
  library(TwoSampleMR)
  library(MRPRESSO)
})
```

In [2]:
```
# Global parameters

# Define the protein that will be analyzed
PROTEIN <- "lrrc37a2"

# LD reference file from 1000 Genomes, region-restricted to PROTEIN's cis fo
BFILE <- file.path("./data", paste0("1000G.EUR.mwe.", PROTEIN))
PLINK_BIN <- "plink"
```

Step 1: Find genetic instruments (overlap protein pQTL with the outcome GWAS)

A variant only qualifies as an instrument here if it's (a) genome-wide significant for the protein, **and** (b) present in the outcome GWAS, so we can compare its effect on both.

We join the two datasets by chromosome:position, then harmonize alleles: a variant only "matches" if the protein GWAS's alleles line up with the outcome GWAS's ref/alt — either directly, or flipped (in which case we sign-flip the protein's beta and reverse its alleles so it's on the same scale).

In [3]:
```
# Read in the protein summary statistics
protein_sumstats <- fread(file.path("./data/", paste0(PROTEIN, "_5e08_signif
# Read in the MVP outcome summary statistics
mvp_sumstats <- fread(file.path("./data/", paste0("MVP_outcome_liftover_", P

# Print a summary of the data
cat(sprintf("Protein: %d variants | MVP GWAS: %d variants\n", nrow(protein_s
```

```
Protein: 6680 variants | MVP GWAS: 6898 variants
```

In [4]:
```
# Build chr:pos join keys to match with the outcome GWAS's ref/alt conventio
protein_sumstats[, SNP_id_0 := paste(CHROM, GENPOS, sep = ":")]
protein_sumstats[, SNP_id_a := paste(CHROM, GENPOS, ALLELE0, ALLELE1, sep =
protein_sumstats[, SNP_id_b := paste(CHROM, GENPOS, ALLELE1, ALLELE0, sep =
mvp_sumstats[, SNP_id_0 := paste(chrom, pos, sep = ":")]

overlap <- inner_join(protein_sumstats, mvp_sumstats, by = "SNP_id_0", relat
cat(sprintf("Overlapping chr:pos positions: %d\n", nrow(overlap)))
```

```
Overlapping chr:pos positions: 5086
```

In [5]:
```
# Harmonize alleles:
# Consistent = protein GWAS's alleles already match the outcome's ref/alt
consistent <- overlap[CHROM == chrom & GENPOS == pos & ALLELE1 == alt & ALLE
consistent[, `:=`(SNP = SNP_id_a, chr = CHROM, position = GENPOS, Beta = BET
                  effect_allele = ALLELE1, other_allele = ALLELE0, eaf = A1
# Flipped = reversed, so sign-flip beta and invert eaf
flipped <- overlap[CHROM == chrom & GENPOS == pos & ALLELE1 == ref & ALLELE0
flipped[, `:=`(SNP = SNP_id_b, chr = CHROM, position = GENPOS, Beta = -BETA,
               effect_allele = ALLELE0, other_allele = ALLELE1, eaf = 1 - A1

overlap_full <- rbindlist(list(consistent, flipped), use.names = TRUE, fill
overlap_full[, Phenotype := PROTEIN]
cat(sprintf("Harmonized instruments (consistent + flipped alleles): %d\n", r
```

```
Harmonized instruments (consistent + flipped alleles): 5083
```

In [6]:
```
# Reformat the exposure data
exposure_data <- overlap_full[, .(SNP, SNP_ID, chr, position, effect_allele,
                                  eaf, samplesize = N, Beta, se, Pval, Pher

# Output exposure data file
fwrite(exposure_data, file.path("./output", paste0(PROTEIN, "_exposure.csv")
head(exposure_data)
```

A data.table: 6 × 12

SNP	SNP_ID	chr	position	effect_allele	other_allele	eaf
<chr>	<chr>	<int>	<int>	<chr>	<chr>	<dbl>
17:45520876:T:TA	rs555712899	17	45520876	TA	T	0.9898011
17:45565380:C:T	rs11652010	17	45565380	T	C	0.9136823
17:45565418:C:G	rs34527929	17	45565418	G	C	0.9370888
17:45573933:A:G	rs559524093	17	45573933	G	A	0.9960206
17:45575763:G:A	rs141461397	17	45575763	A	G	0.9658983
17:45577102:T:C	rs62064364	17	45577102	C	T	0.7788240

Step 2: LD clumping (keep only independent instruments)

For the protein's candidate instruments (N=5,083 harmonized cis pQTL overlapping MVP, from Step 1), LD clumping is performed to obtain a set of LD-independent candidate instruments.

In [7]:

```
clump_input <- overlap_full[, .(rsid = SNP_ID, pval = Pval, SNP = SNP)] %>%
  distinct(rsid, .keep_all = TRUE)
cat(sprintf("Candidate variants for clumping: %d\n", nrow(clump_input)))
```

```
Candidate variants for clumping: 5083
```

In [8]:

```
clumped <- ld_clump(
  dat = clump_input[, c("rsid", "pval")],
  clump_kb = 250,
  clump_r2 = 0.10,
  clump_p = 1e-4,
  bfile = BFILE,
  plink_bin = PLINK_BIN
)
cat(sprintf("Independent instruments retained: %d\n", nrow(clumped)))
retained_snp <- clump_input$SNP[match(clumped$rsid, clump_input$rsid)]
exposure_clumped <- exposure_data %>% filter(SNP %in% retained_snp)

# Output
fwrite(exposure_clumped, file.path("./output/", paste0("MVP_", PROTEIN, "_ex
head(exposure_clumped)
```

```
Clumping UGn4eI, 5083 variants, using: ./data/1000G.EUR.mwe.lrrc37a2

Removing 4967 of 5083 variants due to LD with other variants or absence from
LD reference panel
```

```
Independent instruments retained: 116
```

A data.table: 6 × 12

SNP	SNP_ID	chr	position	effect_allele	other_allele	eaf
<chr>	<chr>	<int>	<int>	<chr>	<chr>	<dbl>
17:45520876:T:TA	rs555712899	17	45520876	TA	T	0.9898011
17:45575763:G:A	rs141461397	17	45575763	A	G	0.9658983
17:45577102:T:C	rs62064364	17	45577102	C	T	0.7788240

These are LD-independent candidate instruments, not yet the final set — see the count Step 2 prints above. The final independent instrument set is determined in Step 3, after harmonizing and reformating the data.

Step 3: Run MR-PRESSO

Prior to running MR-PRESSO, preprocessing the data is needed for MR-PRESSO input. First, reformat the outcome GWAS into a standard schema. Then `harmonise_data()` aligns each instrument's exposure and outcome effect estimates onto the same effect allele (an instrument can be excluded here too, e.g. if it's palindromic and allele frequency can't resolve the strand or missing information for a variant). Finally, `MRPRESSO::mr_presso()` runs three tests on the harmonized instruments:

- **Global test**
- **Outlier test**
- **Distortion test**

More information about the different tests can be found in the introduction of this exercise

Preprocessing

First reformat the data for MR-PRESSO input, next harmonize the LD-independent candidate instruments aginst the outcome GWAS.

In [9]:

```
# Check the effect allele column
stopifnot("Expected ea == alt for every row (beta is assumed to be on the al
            all(mvp_sumstats$ea == mvp_sumstats$alt))

# Reformat the outcome GWAS to a standard schema
outcome_data <- mvp_sumstats[, .(
  SNP = paste(chrom, pos, ref, alt, sep = ":"),
  CHR = chrom, pos = pos, ref = ref, alt = alt,
  af_alt = af, beta = beta, sebeta = sebeta, Pval = pval,
  N = num_samples, Phenotype = "SNHL"
)]
```

```
fwrite(outcome_data, file.path("./output/", paste0("MVP_outcome_", PROTEIN,
cat(sprintf("Outcome variants: %d\n", nrow(outcome_data)))
```

```
Outcome variants: 6898
```

In [10]:
```
# Harmonize exposure + outcome onto the same effect allele
exposure_fmt <- format_data(
  as.data.frame(exposure_clumped), type = "exposure",
  snp_col = "SNP", pos_col = "position", beta_col = "Beta", se_col = "se",
  effect_allele_col = "effect_allele", other_allele_col = "other_allele",
  eaf_col = "eaf", pval_col = "Pval", samplesize_col = "samplesize", phenoty
)

outcome_fmt <- format_data(
  as.data.frame(outcome_data), type = "outcome", snps = exposure_fmt$SNP,
  snp_col = "SNP", chr_col = "CHR", beta_col = "beta", se_col = "sebeta",
  effect_allele_col = "alt", other_allele_col = "ref", eaf_col = "af_alt",
  pval_col = "Pval", samplesize_col = "N", phenotype_col = "Phenotype"
)

dat <- harmonise_data(exposure_fmt, outcome_fmt, action = 2)
dat_filtered <- dat[dat$mr_keep == TRUE, ]
# Reset row names after filtering: base R keeps the pre-filter row labels (e
# skipping any dropped row), which desyncs them from row *position*. mr_pres
# indices are positional, so without this reset, indices reported by the Dis
# ("Outliers Indices") can point to the wrong row if later cross-checked by
# of position.
rownames(dat_filtered) <- NULL
cat(sprintf("Harmonized instruments for MR: %d (of %d clumped)\n", nrow(dat_

stopifnot("Need >= 3 harmonized instruments for MR-PRESSO" = nrow(dat_filter
```

```
Warning message in format_data(as.data.frame(outcome_data), type = "outcom
e", snps = exposure_fmt$SNP, :
“The following SNP(s) are missing required information for the MR tests and
will be excluded
17:46880656:t:c”
Harmonising lrrc37a2 (9Ng010) and SNHL (122UPg)
```

```
Harmonized instruments for MR: 115 (of 116 clumped)
```

This is the final independent instrument set (N = 115 variants) that will be used for the MR-PRESSO input below

Run MR-PRESSO

This step may take about 3-5 minutes to run.

In [11]:
```
# Run MR-PRESSO
mr_presso_result <- mr_presso(
  BetaOutcome = "beta.outcome", BetaExposure = "beta.exposure",
  SdOutcome = "se.outcome", SdExposure = "se.exposure",
  OUTLIERtest = TRUE, DISTORTIONtest = TRUE,
```

```
  data = dat_filtered, NbDistribution = 3000, SignifThreshold = 0.05
)
```

Causal estimate (not a test)

The IVW (inverse-variance-weighted) regression result providing a raw estimate, sd, and p-value across all instruments with no outlier removal. This is `mr_presso()`'s output table, not one of its three tests.

In [12]:
```
main_results <- mr_presso_result$`Main MR results`
write.csv(main_results, file.path("./output/", paste0("MVP_", PROTEIN, "_mr_
main_results
```

A data.frame: 2 × 6

Exposure	MR Analysis	Causal Estimate	Sd	T-stat	P-value
<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
beta.exposure	Raw	0.009043356	0.002562829	3.528662	0.0006032323
beta.exposure	Outlier-corrected	0.009542365	0.002476918	3.852515	0.0001944485

Global test

The global test detects horizontal pleiotropy by evaluating whether the variability across instrument-specific causal estimates is greater than expected by chance.

In [13]:
```
global_test <- mr_presso_result$`MR-PRESSO results`$`Global Test`
cat(sprintf("RSSobs = %.3f, p-value = %s\n", global_test$RSSobs, global_test
```

```
RSSobs = 215.865, p-value = <0.000333333333333333
```

Outlier test

The outlier test corrects for horizontal pleiotropy by removing specific SNPs. It uses per-instrument p-values to identify which genetic variants are driving the excess variability in the global test.

In [14]:
```
outlier_test_raw <- mr_presso_result$`MR-PRESSO results`$`Outlier Test`
  if (is.data.frame(outlier_test_raw)) {
    outlier_test <- outlier_test_raw
    stopifnot("Outlier test row count doesn't match main results - can't saf
              nrow(outlier_test) == nrow(dat_filtered))
    outlier_test$SNP <- dat_filtered$SNP
    outlier_test$Pvalue <- as.numeric(gsub("<", "", outlier_test$Pvalue))
    write.csv(outlier_test, file.path("./output/", paste0("MVP_", PROTEIN, "
    cat(sprintf("%d of %d instruments flagged as outliers (p < 0.05)\n", sum
    outlier_test[outlier_test$Pvalue < 0.05, ]
  } else {
```

```
    cat("Outlier test not computed — global test wasn't significant enough t
  }
```

```
1 of 115 instruments flagged as outliers (p < 0.05)
```

A data.frame: 1 × 3

	RSSobs	Pvalue	SNP
	<dbl>	<dbl>	<chr>
45	0.0009209804	0.03833333	17:46008244:t:c

Distortion test

The distorition test estimates the variability in the casual estimate once flagged outliers are removed — the gap between the raw and outlier-corrected estimates in the Main MR results above.

In [16]:
```
distortion_test_raw <- mr_presso_result$`MR-PRESSO results`$`Distortion Test
if (!is.null(distortion_test_raw)) {
  outlier_indices <- distortion_test_raw$`Outliers Indices`
  outlier_snps <- if (is.numeric(outlier_indices)) paste(dat_filtered$SNP[ou
  distortion_test <- data.frame(
    Outlier_Indices = paste(outlier_indices, collapse = ","),
    Outlier_SNPs = outlier_snps,
    Distortion_Coefficient = distortion_test_raw$`Distortion Coefficient`,
    Pvalue = distortion_test_raw$Pvalue
  )
  write.csv(distortion_test, file.path("./output/", paste0("MVP_", PROTEIN,
  distortion_test
} else {
  cat("Distortion test not computed — no outliers were flagged, so there's r
}
```

A data.frame: 1 × 4

	Outlier_Indices	Outlier_SNPs	Distortion_Coefficient	Pvalue
	<chr>	<chr>	<dbl>	<dbl>
beta.exposure	45	17:46008244:t:c	-5.229401	0.8173333

Step 4: Post-process — pick the final estimate

The production pipeline's last step merges every protein's `main_results.csv` into one table, and for each protein picks a single final causal estimate: the **outlier-corrected** estimate if MR-PRESSO computed one (i.e. outliers were found and removed), otherwise the **raw** estimate. That's the number that actually gets reported/interpreted downstream — everything above (global/outlier/distortion tests) exists to justify *which* row this picks.

Here there's just one protein, so "merging" collapses to picking that one row — but it's the same decision rule and the same output schema (`Protein,Exposure,Estimate,Sd,P`) as the production merge, so this row is directly comparable to (or droppable into) that larger table.

In [17]:
```
raw_row <- main_results[main_results$`MR Analysis` == "Raw", ]
outlier_row <- main_results[main_results$`MR Analysis` == "Outlier-correctec

# Use the outlier-corrected estimate if it exists and is valid
final_row <- if (nrow(outlier_row) > 0 && !is.na(outlier_row$`P-value`[1]))
cat(sprintf("Using the '%s' estimate\n", final_row$`MR Analysis`[1]))

final_result <- data.frame(
  Protein = PROTEIN,
  Exposure = final_row$Exposure[1],
  Estimate = final_row$`Causal Estimate`[1],
  Sd = final_row$Sd[1],
  P = final_row$`P-value`[1]
)

write.csv(final_result, file.path("./output/", paste0("SNHL_MVP_", PROTEIN,
head(final_result)
```

```
Using the 'Outlier-corrected' estimate
```

A data.frame: 1 × 5

	Protein	Exposure	Estimate	Sd	P
	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	lrrc37a2	beta.exposure	0.009542365	0.002476918	0.0001944485